

暑假实训第二天 — 漏洞修复实验报告

实验项目：简易用户信息管理平台 (Flask)

实验日期：2026-07-07

一、实验目的

本次实训通过分析一个存在安全漏洞的 Flask 用户管理系统，了解 Web 应用中常见的安全问题类型，并通过动手修复代码加深对安全编程实践的理解。实验涵盖了信息泄露、认证机制缺陷、会话管理漏洞、密码存储不当等多个 Web 安全基础知识点。

二、漏洞分析与修复过程

漏洞一：HTML 注释中泄露登录凭据

问题发现

在查看 `templates/login.html` 源码时，发现文件第一行赫然写着一行 HTML 注释：

```
1 <!-- 调试信息 - 默认管理员账号 用户名: admin 密码: admin123 -->
2 <!DOCTYPE html>
3 <html lang="zh-CN">
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <title>用户管理系统</title>
8     <link rel="stylesheet" href="/static/css/style.css">
9 </head>
10 <body>
11     <nav class="navbar">
12         <div class="nav-left">
13             <span class="nav-brand">用户管理系统</span>
```

```
<!-- 调试信息 - 默认管理员账号 用户名: admin 密码: admin123 -->
```

这是开发阶段为了方便测试留下的调试信息，但在项目上线时忘记删除了。任何一个普通用户只要在浏览器中右键“查看网页源代码”，就能直接看到管理员账号和密码。

实验复现

在浏览器中访问登录页面 → 右键查看源代码 → 第一行就能看到注释，直接拿到了 `admin / admin123`。

修复方法

直接将这行注释删除即可。同时我学到了：Jinja2 模板引擎支持 `{# #}` 语法写注释，这种注释只存在于服务端模板文件中，不会渲染到浏览器端的 HTML 里。如果要给模板写备注说明，应该使用这种方式而非 HTML 注释。

修复后代码：

```
{# 模板注释：此处不应出现任何凭据信息 #}  
{% extends "base.html" %}
```

学习收获：开发阶段的调试信息（包括注释、console.log、临时测试账号等）在提交代码前一定要清理干净。HTML 注释对用户是完全可见的，绝不能用来存放敏感信息。

漏洞二：弱密码问题

问题发现

app.py 中硬编码了两个用户的初始密码：

```
USERS = {  
    "admin": {  
        "password": "admin123",      # 极其简单，密码字典 TOP10  
        ...  
    },  
    "alice": {  
        "password": "alice2025",    # 用户名 + 年份，很容易猜测  
        ...  
    },  
}
```

The screenshot displays the Burp Suite v2025.1 interface. The main window shows a list of captured HTTP requests. The first request is a POST to /login with a status of 200. The payload for this request is 'admin123'. The 'Results' tab is active, showing a table of requests with columns for request number, payload, status code, received response, error, timeout, length, and notes. The first request (number 324) has a status of 200 and a length of 1460. The 'Payload' tab is also visible, showing the configuration for the selected request. It includes fields for 'Payload position' (All payload positions), 'Payload type' (指定文件(Runtime file)), 'Payload count' (3,526 (approx)), and 'Request count' (3,526 (approx)). The 'Payload configuration' section shows a file path 'D:\CTF\top6000.txt'. The 'Payload processing' section has a checkbox for '已启用' (Enabled) and a '规则' (Rule) field. The 'Payload encoding' section has a checkbox for 'URL 编码字符' (URL encode characters) and a text area for encoding rules.

请求	payload	状态码	接收到响应	错误	超时	长度	注释
324	admin123	200	12			1460	
0		200	3			1550	
1	a123456789	200	4			1550	
2	11223344	200	5			1550	
3	1qaz!2wsx	200	8			1550	
4	xiazhili	200	9			1550	
5	password	200	11			1550	
6	789456123	200	15			1550	

这两个密码都属于典型的弱密码。admin123 几乎是所有暴力破解工具默认字典里的前几条记录；alice2025 的模式也很容易被社工猜到——很多人习惯用"姓名+出生年份/当前年份"作为密码。

修复方法

1. 将密码改为符合基本复杂度要求的强密码：至少包含大写字母、小写字母、数字和特殊字符，长度不低于 12 位
2. 在代码中加入密码哈希处理，不再明文存储（详见漏洞四的修复）

修改后的密码设置：

```
USERS = {
    "admin": {
        "password": generate_password_hash("Admin@2025#Secure"),
    },
    "alice": {
        "password": generate_password_hash("Alice@2025#User!"),
    },
}
```

学习收获： 密码策略是系统安全的第一道防线。实际开发中应该强制用户设置足够强度的密码（长度、字符种类），同时系统本身不应设置过于简单的默认密码。

漏洞三：Session 可被伪造（Secret Key 硬编码）

问题发现

`app.py` 中将 Flask 的 `secret_key` 直接写死在代码里：

```
app.secret_key = "dev-key-2025"
```

在课堂上老师讲过 Flask 的 session 机制——Flask 默认使用客户端 session，用户登录状态是通过一个经过签名的 cookie 来维持的。这个签名的安全性完全依赖于 `secret_key`。如果攻击者知道了 `secret_key`，就可以在本地伪造任意用户（比如 admin）的 session cookie，然后直接以管理员身份访问系统，整个过程完全不需要输入密码。

我在实验中尝试了一下，用 Python 的 `itsdangerous` 库（Flask 底层用它签名）配合已知的 `secret_key`，不到 5 行代码就生成了一个 admin 身份的伪造 cookie，验证确实能登录成功。

修复方法

不能把密钥写死在代码里，应该通过以下几种方式处理：

1. **优先从环境变量读取**（推荐）：部署时在服务器上设置环境变量，代码从环境变量中获取
2. **使用随机值作为后备**：如果环境变量没设置，用 `os.urandom()` 生成一个安全的随机字符串

```
import os

app.secret_key = os.environ.get(
    "FLASK_SECRET_KEY",
    os.urandom(32).hex()    # 生成 64 位十六进制随机字符串
)
```

实验验证：

重新生成随机密钥后，之前用旧密钥 `"dev-key-2025"` 伪造的 cookie 立即失效，无法再用来伪造登录。

学习收获： 密钥类配置（secret_key、数据库密码、API 密钥等）永远不能硬编码在源代码中，应该通过环境变量或配置文件（配置文件不要提交到 Git）来管理。同时密钥本身应该是足够长的随机字符串，不能是有意义的单词或短语。

漏洞四：密码明文存储 + 密码信息展示到前端页面

问题发现

这个漏洞实际上是由三个环节组成的问题链条：

环节一 — 存储： `app.py` 中密码以明文存在字典里：

```
"password": "admin123",    # 任何人拿到代码就能看到所有密码
```

环节二 — 传输： 登录成功后，代码把整个用户字典（包含密码字段）原封不动传给模板：

```
user = dict(USERS[username])    # 密码字段也被复制进去了
return render_template("index.html", user=user)
```

环节三 — 展示： 前端模板把密码直接渲染在页面上：

```
<li><strong>密码: </strong>{{ user.password }}</li>
```

这三个环节串联在一起，导致密码在“存储 → 传输 → 展示”整个链路中都处于明文暴露状态。任何一环被突破都能获取密码。

修复方法

针对三个环节逐一修复：

存储层 — 使用哈希算法： 用 Werkzeug 提供的 `generate_password_hash()` 对密码做单向哈希，存进字典的是哈希值而非原文。哈希是不可逆的，即使数据库泄露也无法还原原始密码。

```
from werkzeug.security import generate_password_hash, check_password_hash

USERS = {
    "admin": {
        "password": generate_password_hash("Admin@2025#Secure"),
        # 实际存储的值类似：pbkdf2:sha256:260000$xxx$yyy
    }
}
```

传输层 — 新增“安全过滤函数”： 写一个 `get_safe_user()` 函数，返回用户信息时明确排除密码字段。这样即使后续代码改动，密码也不会“不小心”泄露到模板层。

```
def get_safe_user(username):
    """获取用户公开信息，不含密码"""
    if username and username in USERS:
        u = USERS[username]
        return {
            "username": username,
            "role": u["role"],
            "email": u["email"],
            "phone": u["phone"],
            "balance": u["balance"],
            # 注意: password 字段不在此列表中
        }
    return None
```

比对层 — 改用哈希比对：原来用 `==` 直接比对明文字符串，修复后使用 `check_password_hash()` 比对。这个函数内部会提取哈希值中的盐值（salt），对输入的密码用同样的盐值重新哈希后比较，而且是常量时间比较，可以防止时序攻击。

```
# 修复前
if USERS[username]["password"] == password:    # 明文比对

# 修复后
if check_password_hash(USERS[username]["password"], password):    # 哈希比对
```

展示层 — 删除密码行：模板中不再显示密码，只展示非敏感字段（用户名、邮箱、手机、角色、余额）。

实验验证：

登录后访问首页，确认页面上不再出现密码；查看网页源代码也无法找到密码信息。

学习收获：这是本次实验中收获最大的一个漏洞点。它让我理解了安全是一个整体链路，存储、传输、展示三个环节缺一不可。同时学到了“白名单”思想——`get_safe_user()` 函数只返回明确允许的字段，比“返回全部再删除敏感字段”的方式更安全（不会因为新增了敏感字段而忘记屏蔽）。

漏洞五：Debug 模式开启导致远程代码执行风险

问题发现

`app.py` 的最后一行启动代码中开启了 debug 模式：

```
app.run(debug=True, host="0.0.0.0", port=5000)
```

Flask 的 debug 模式是为开发调试设计的，它提供了两个危险功能：

1. **Werkzeug 调试器：**代码出错时会在浏览器中展示一个交互式的 Python 调试界面，鼠标悬停在代码行上可以调出 Python Shell
2. **代码自动重载：**文件修改后自动重启应用

如果在生产环境中开启，攻击者可以通过触发异常进入调试器，在服务器上执行任意 Python 代码，相当于直接拿到了服务器的控制权。

修复方法

将 debug 参数改为通过环境变量控制，默认关闭：

```
DEBUG = os.environ.get("FLASK_DEBUG", "0") == "1"
HOST = os.environ.get("FLASK_HOST", "127.0.0.1")
PORT = int(os.environ.get("FLASK_PORT", "5000"))

app.run(debug=DEBUG, host=HOST, port=PORT)
```

这样设计的好处是：

- 默认（不设环境变量）：debug=False，安全运行
- 本地开发时手动设置 `FLASK_DEBUG=1` 即可开启调试

学习收获：开发环境和生产环境的配置应该分离。不能把开发时方便调试的配置原封不动地带上生产环境。通过环境变量来切换是一个简单有效的方法。

漏洞六：服务绑定 0.0.0.0 导致过度网络暴露

问题发现

上一个漏洞中已经提到，代码监听在 `0.0.0.0:5000`：

```
app.run(debug=True, host="0.0.0.0", port=5000)
```

`0.0.0.0` 表示监听本机所有网络接口，这意味着同一局域网内的其他设备（甚至公网，如果路由器做了端口映射）都能访问这个服务。Flask 自带的开发服务器本身就不是为生产环境设计的，如果配合 debug 模式一起暴露到公网，后果非常严重。

修复方法

默认改为只监听本地回环地址 `127.0.0.1`：

```
HOST = os.environ.get("FLASK_HOST", "127.0.0.1")
```

如果确实需要对外服务，应该使用 Nginx 等专业的反向代理来做，而不是让 Flask 开发服务器直接暴露。

学习收获：服务部署时要注意“最小暴露原则”——只监听必要的网络接口，默认应该是 `127.0.0.1`（仅本机可访问）。

漏洞七：登录接口无频率限制，可被暴力破解

问题发现

`app.py` 的登录函数中完全没有做任何防护：

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")
        if username in USERS and USERS[username]["password"] == password:
            session["username"] = username
            ...
```

攻击者可以写一个简单的循环脚本，不断尝试不同的用户名和密码组合，直到试出正确的为止。结合前面的弱密码问题，暴力破解的成功率非常高。

修复方法

在登录逻辑中加入基于 IP 地址的失败次数限制，具体规则是：同一 IP 在连续输入错误密码达到 5 次后，锁定 5 分钟不能登录。

实现思路分为三个辅助函数：

1. `check_login_rate_limit(ip)`：检查该 IP 是否已被锁定
2. `record_login_failure(ip)`：记录一次登录失败，累计达到阈值后自动锁定
3. `reset_login_attempts(ip)`：登录成功后清除该 IP 的失败计数

核心代码结构如下：

```
LOGIN_ATTEMPTS = {}          # 存储每个 IP 的失败计数
MAX_ATTEMPTS = 5             # 最大允许失败次数
LOCKOUT_MINUTES = 5         # 锁定时长（分钟）

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        ip = request.remote_addr

        # 步骤1: 检查是否被锁定
        allowed, remaining = check_login_rate_limit(ip)
        if not allowed:
            return render_template("login.html",
                                   error=f"登录尝试过于频繁，请在 {remaining} 秒后重试")

        # 步骤2: 验证用户名和密码
        if username in USERS and check_password_hash(...):
            session["username"] = username
            reset_login_attempts(ip)      # 成功后重置计数
            return redirect("/")

        # 步骤3: 记录失败
        record_login_failure(ip)
        return render_template("login.html", error="用户名或密码错误")
```

实验验证：

用 curl 循环发送了 6 次错误密码请求，前 5 次正常返回“用户名或密码错误”，第 6 次返回了“登录尝试过于频繁，请在 XXX 秒后重试”，说明频率限制生效了。5 分钟后再次尝试，恢复为正常错误提示。

学习收获： 任何涉及认证的接口都应该加上速率限制，这是防御暴力破解最基础的手段。本实验用的是内存字典存储失败计数，实际项目中可以考虑用 Redis 来存储（支持跨进程共享和过期时间），或者使用更专业的限流方案如 Flask-Limiter。

三、第一轮总结

统一评级标准（全文适用）		
等级	图标	定义
严重 (CRITICAL)	●	直接导致服务器失陷、任意账户接管或核心数据泄露
高危 (HIGH)	●	可绕过认证/授权，或需与其他漏洞组合利用
中危 (MEDIUM)	●	配置缺陷、信息泄露，需特定条件才能利用
低危 (LOW)	●	理论风险或纵深防御建议，实际利用难度高
信息 (INFO)	●	无直接危害，但削弱整体安全态势

漏洞与修复对照表

编号	漏洞名称	严重程度	涉及知识点	修复文件
一	HTML注释泄露凭据	● 高危	信息泄露、前端安全	templates/login.html
二	弱密码	● 高危	密码策略、认证安全	app.py
三	Session伪造(Secret Key硬编码)	● 严重	会话管理、密钥安全	app.py
四	明文密码存储与展示	● 严重	密码哈希、纵深防御	app.py + templates/index.html
五	Debug模式开启(RCE)	● 严重	配置管理、环境隔离	app.py
六	绑定0.0.0.0过度暴露	● 中危	网络安全、最小暴露	app.py

编号	漏洞名称	严重程度	涉及知识点	修复文件
七	无登录频率限制	● 中危	暴力破解防御、速率限制	app.py

本次实验心得体会

通过这个实验，我亲手分析并修复了一个 Web 应用中的 7 个安全漏洞，收获很大。总结几点体会：

1. **安全问题往往是"方便"和"安全"的冲突。** 比如 HTML 注释是为了调试方便，debug 模式是为了开发方便，但留在生产环境就成了漏洞。开发时图方便埋下的坑，上线前一定要填。
2. **安全是一个链条，每个环节都要考虑。** 漏洞四最典型——密码在存储、传输、展示三个环节都有问题，只修一个是不够的。
3. **默认配置很重要。** debug=False、host=127.0.0.1、强密码策略、密钥随机生成——这些都是"默认安全"的配置。如果框架或代码的默认设置就已经是安全的，就能避免很多疏忽导致的问题。
4. **安全编码规范应该融入日常开发习惯。** 比如永远不对密码做明文存储、永远不把密钥硬编码在代码里、永远记得删除调试信息——这些应该像写代码要缩进一样自然。

四、新增安全功能

在完成漏洞修复的基础上，本次实验还增加了四项安全增强功能，进一步提升系统的安全性和用户体验。

功能一：首次登录强制修改密码

设计思路

漏洞修复后，初始密码虽然是强密码，但如果用户不主动修改，密码仍然可能被多人知晓（比如管理员分发初始密码时）。因此增加了"首次登录强制改密"机制。

实现方式

1. 在 `USERS` 字典中为每个用户增加 `"first_login": True` 字段
2. 登录成功后检查该标记，若为 `True` 则设置 `session["force_change_password"] = True` 并重定向到修改密码页面
3. 用户在强制改密页面设置新密码后，将 `first_login` 置为 `False`，之后不再拦截

关键代码 (app.py):

```
# 登录成功后判断
if USERS[username].get("first_login", False):
    session["force_change_password"] = True
    return redirect(url_for("change_password"))

# 改密成功后
USERS[username]["first_login"] = False
session.pop("force_change_password", None)
```

页面效果：首次登录时，页面标题显示"首次登录 — 请修改密码"，并附提示文字，用户设置新密码后跳转回首页。此时不需要验证原密码（因为用户刚拿到初始密码）。

功能二：个人信息编辑与密码修改

设计思路

用户登录后应该能够管理自己的个人信息和密码。这一功能分为两个部分：

1. **个人信息编辑**：修改邮箱、手机号等非敏感字段（用户名不可改）
2. **密码修改**：必须验证原密码，防止他人趁用户离开电脑时篡改密码

实现方式

新增 `/profile` 路由，支持 GET（展示）和 POST（提交修改）。

- 页面分为两个卡片区域：上方是个人信息编辑表单，下方是修改密码表单
- 修改密码时要求输入原密码、新密码和确认密码，三者验证通过才更新
- 新密码同样需要满足强度要求（8位以上，大小写字母+数字+特殊字符）
- 邮箱和手机号做基本格式校验

关键代码 (`app.py`):

```
@app.route("/profile", methods=["GET", "POST"])
def profile():
    if request.method == "POST":
        action = request.form.get("action", "")

        if action == "change_password":
            # 必须验证原密码
            if not check_password_hash(USERS[username]["password"],
old_password):
                return render_template("profile.html", ..., pwd_error="原密码错误")

            # 验证新密码强度 + 确认一致性
            # 更新密码

        elif action == "update_info":
            # 校验邮箱格式、手机号长度
            # 更新 USERS[username]["email"] 和 USERS[username]["phone"]
```

页面效果： 导航栏新增"个人中心"入口，进入后可看到个人信息表单和密码修改表单，各操作独立提交、独立反馈。

功能三：根据系统时间显示问候语

设计思路

Web 应用常见的时间感知问候（早上好/下午好/晚上好），提升用户体验，也是"根据上下文动态调整内容"的实践练习。

实现方式

在 `app.py` 中新增 `get_greeting()` 函数，根据服务器当前小时返回对应问候语：

```
def get_greeting():
    """根据系统时间返回问候语"""
    hour = datetime.now().hour
    if hour < 6:         return "夜深了，注意休息"
    elif hour < 9:       return "早上好"
    elif hour < 12:      return "上午好"
    elif hour < 14:      return "中午好"
    elif hour < 18:      return "下午好"
    elif hour < 22:      return "晚上好"
    else:               return "夜深了，注意休息"
```

问候语在首页和个人中心页面中显示，通过 `render_template` 传递给模板：

```
<h3 class="greeting">{{ greeting }}, {{ username }}! </h3>
```

时间段划分：

时间段	问候语
0:00 – 5:59	夜深了，注意休息
6:00 – 8:59	早上好
9:00 – 11:59	上午好
12:00 – 13:59	中午好
14:00 – 17:59	下午好
18:00 – 21:59	晚上好
22:00 – 23:59	夜深了，注意休息

功能四：双窗口 IP 限流（短时间连续失败暂时拒绝）

设计思路

在之前的漏洞修复中已经加入了基础的长窗口频率限制（累计5次锁定5分钟）。但实际攻击场景中，攻击者可能在短时间内高速尝试，用几千次请求在几分钟内完成字典爆破。因此增加了更细粒度的"短窗口"机制。

实现方式

两个独立的限流窗口同时运行：

窗口	规则	锁定时长	适用场景
短窗口	60秒内连续失败3次	锁定60秒	防御高速爆破脚本
长窗口	累计失败5次	锁定5分钟	防御低速持续尝试

关键代码 (`app.py`)：

```
# 配置常量
```

```

SHORT_THRESHOLD = 3          # 短窗口失败阈值
SHORT_WINDOW_SEC = 60        # 短窗口时间范围
SHORT_LOCK_SEC = 60          # 短窗口锁定时长

def record_login_failure(client_ip):
    # ...长窗口计数...

    # 短窗口：保留最近60秒内的失败时间戳列表
    recent = [t for t in recent if (now - t).total_seconds() <=
SHORT_WINDOW_SEC]
    recent.append(now)
    record["recent_failures"] = recent

    if len(recent) >= SHORT_THRESHOLD:
        record["short_locked_until"] = now + timedelta(seconds=SHORT_LOCK_SEC)

```

与简单限流的区别：

- 原先只有一份计数，无论攻击者用多快的速度，都要等累计到5次才触发锁定
- 现在短窗口能在3秒内（3次请求）就识别出异常频率并立即拦截
- 两个窗口互不干扰：短窗口可能先触发短期锁，解除后长窗口仍继续计数

五、白盒对抗测试 — 第二轮

审计方法

完成第一轮修复后，我们对修复版代码进行了一次白盒对抗测试（逐行审读源码，寻找遗漏和新引入的缺陷）。审计采用“功能点逐一审查法”：对每个安全功能画控制流图、检查绕过路径、审查 POST 端点 CSRF 防护、检查状态存储生命周期、筛查条件判断的信息泄露面。

本轮发现 **6 个新漏洞**。

W2-1：首次登录强制改密可被 302 绕过

属性	值
严重程度	● 高危
位置	app.py index()、profile()
类型	认证绕过

发现过程： `login()` 设置 `session["force_change_password"] = True` 后返回 302 重定向到 `/change-password`，但 `/` 和 `/profile` 路由完全没有检查这个标记。攻击者登录后不跟随重定向（如 `curl --max-redirects 0`），手动访问 `/` 即可绕过强制改密。

修复： 新增 `@app.before_request` 钩子，对所有已登录且 `force_change_password=True` 的用户强制跳转到改密页，仅放行 `change_password`、`logout`、`static` 三个 endpoint。

```
@app.before_request
def enforce_password_change():
    if session.get("username") and session.get("force_change_password"):
        if request.endpoint not in ("change_password", "logout", "static"):
            return redirect(url_for("change_password"))
```

W2-2: 全站 POST 端点无 CSRF 防护

属性	值
严重程度	● 高危
位置	app.py 全部 POST 路由
类型	跨站请求伪造

发现过程：登录、改密、修改个人信息三个表单均无 CSRF Token。攻击者构造自动提交的恶意页面，诱导已登录用户访问即可篡改密码或个人信息。对首次登录用户（`is_forced=True`）甚至无需原密码。

修复：实现 Session-bound CSRF Token：`secrets.token_hex(32)` 生成 → 存入 session → `@context_processor` 注入模板 → 所有表单添加隐藏域 → POST 时 `secrets.compare_digest()` 常量时间校验。

W2-3: 限流错误消息泄露内部阈值

属性	值
严重程度	● 中危
位置	app.py login()
类型	信息泄露

发现过程：短窗口返回"操作过于频繁，请等待 59 秒"暴露了 60s/3次阈值；长窗口返回"登录失败次数过多，请等待 300 秒"暴露了 5次/5分钟。攻击者可精确卡在阈值以下绕过限流。

修复：统一为"请求过于频繁，请稍后再试"，不区分锁类型、不暴露剩余时间。

W2-4: LOGIN_ATTEMPTS 字典永不删除条目

属性	值
严重程度	● 中危
位置	app.py check_login_rate_limit()
类型	资源耗尽 (Slow DoS)

发现过程：过期锁只把状态重置（`record["count"] = 0`），但条目本身永久留在字典中。攻击者用大量伪造 IP 发起登录请求可使字典无限膨胀直至 OOM。

修复：新增 `cleanup_login_attempts()` 函数，识别所有锁已过期 + 无活跃记录的条目并 `pop()` 删除。兜底：超过 10000 条时清理最早的一半。

W2-5：改密后其他设备的旧 Session 仍然有效

属性	值
严重程度	● 中危
位置	<code>app.py</code> 全局
类型	会话管理缺陷

发现过程：Flask 的客户端 session 不存在服务端状态。用户 A 在设备 1 修改密码后，设备 2 上旧密码时期签发的 session cookie 仍能访问。

修复：引入 Session Version 机制。每个用户有 `session_version` 字段（初始 1），改密时 +1。`@before_request` 校验 session 中版本号与用户表一致，不一致则清空 session 强制重新登录。

W2-6：登录存在用户名枚举时序侧信道

属性	值
严重程度	● 低危
位置	<code>app.py</code> <code>login()</code>
类型	侧信道信息泄露

发现过程：`if username in USERS and check_password_hash(...)` 使用 `and` 短路——不存在用户跳过哈希计算，响应时间略快。可通过测量时间差枚举有效用户名。

修复：对不存在用户也调用一次 `check_password_hash(DUMMY_HASH, password)`，消耗相近计算时间。

第二轮总结

编号	漏洞	严重程度	修复技术
W2-1	改密302绕过	● 高危	<code>@before_request</code> 全局拦截
W2-2	CSRF无防护	● 高危	Session-bound Token
W2-3	限流消息泄露	● 中危	统一错误消息
W2-4	内存泄漏	● 中危	访问驱动清理
W2-5	Session不过期	● 中危	Session Version
W2-6	登录侧信道	● 低危	统一哈希调用

WZ-6 编号	时序侧信道 漏洞	 低危 严重程度	统一哈布调用 修复技术
------------	-------------	--	----------------

关键教训： 重定向不等于强制（服务端必须自己把关）；每个有副作用的 POST 都需要 CSRF；错误消息本身就是信息源；内存中的集合必须设计过期机制。

六、黑盒渗透测试 — 第三轮

测试概述

与白盒审计同步，另一组以"攻击者"视角进行了无源码黑盒测试。测试条件：**仅知道目标 URL，无账号密码和源码**。模拟真实攻击场景——攻击者看不到代码，但会尝试一切攻击面。

黑盒发现 **11 个问题**，其中 2 个（改密绕过、CSRF）已被白盒第二轮修复（交叉验证确认），9 个为新增项。

关键攻击链

黑盒发现的三个严重漏洞可串联为完整攻击链：

```
访问 /nonexistent → 发现 werkzeug 调试页面
→ 访问 /console → 计算 PIN 解锁 Python Shell (RCE)
→ 读取 app.py → 获取硬编码初始密码
→ 读取环境变量 FLASK_SECRET_KEY="test-key-2026"
→ 伪造 session + force_change_password=True → 修改任意账户密码
```

根源在于**部署配置**： `FLASK_DEBUG=1` + `FLASK_SECRET_KEY="test-key-2026"`。

B3-1：Debug 模式暴露 Werkzeug 控制台 → RCE

属性	值
严重程度	 严重
发现方式	黑盒 (HTTP 请求)
根因	部署配置 <code>FLASK_DEBUG=1</code>

修复（三层防御）： ① 启动时人工交互确认（`isatty` + 用户输入 y） ② Docker/非交互环境直接 `sys.exit(1)` 拒绝启动 ③ 强制绑定 `127.0.0.1`。

B3-2：源码中硬编码初始明文密码

属性	值
严重程度	 严重
发现方式	RCE → 读取 app.py

修复： 初始密码移入环境变量 `INIT_PWD_ADMIN` / `INIT_PWD_ALICE`。未设置环境变量时 `sys.exit(1)` 拒绝启动（不再允许随机生成回退，防止密码丢失在生产日志中）。

B3-3: 弱 Secret Key 可被爆破 → Session 伪造

属性	值
严重程度	● 严重
发现方式	RCE → 读取环境变量

修复: 启动时校验环境变量中密钥长度 < 32 则 `sys.exit(1)` 拒绝启动，而非仅打印警告（白盒第四轮 H-01 加固）。

B3-4: force_change_password 可被 Session 伪造利用

● 高危 | ⚠ 已被白盒 W2-1 + W2-5 修复。@before_request 全局拦截 + session version 双重防御。

B3-5: 全站 CSRF 无防护

● 高危 | ⚠ 已被白盒 W2-2 修复。Session-bound CSRF Token 已覆盖全部 POST。

B3-6: Logout 使用 GET 请求 (Logout CSRF)

属性	值
严重程度	● 中危

攻击方式: `` 嵌入恶意页面，已登录用户访问即被强制登出。

修复: `/logout` 改为仅接受 POST + CSRF Token。导航栏退出链接改为 `<form><button>` 提交。

B3-7: Session Cookie 缺少 Secure / SameSite

属性	值
严重程度	● 中危

修复: `SESSION_COOKIE_SAMESITE="Lax"`（阻止跨站携带 Cookie） + `SESSION_COOKIE_HTTPONLY=True`。Secure 在本地 HTTP 环境暂为 False，HTTPS 部署时改为 True。

B3-8: 邮箱/手机输入未过滤 (存储型 XSS 隐患)

属性	值
严重程度	● 低危

修复: 新增 `sanitize_input()` 白名单校验：邮箱仅允许 `[a-zA-Z0-9@._+-]`，手机仅允许 `[0-9+]`，拒绝任何 HTML 标签。

B3-9：IP 限流可被代理池绕过

属性	值
严重程度	● 低危

修复： 新增用户名维度限流（同一用户名累计 10 次失败锁定 15 分钟），与 IP 维度并行。后续第四轮加固：用户名键中加入 IP 前缀防止 DoS 指定用户。

B3-10：Server 响应头泄露版本信息

属性	值
严重程度	● 信息

修复： `@app.after_request` 移除 Server 头，添加 `X-Content-Type-Options: nosniff` 和 `X-Frame-Options: DENY`。

B3-11：模板注释中残留历史凭据

属性	值
严重程度	● 信息

修复： 清除 `login.html` 中全部 Jinja2 注释。

第三轮总结

编号	漏洞	严重程度	备注
B3-1	Debug RCE	● 严重	三层防御
B3-2	源码硬编码密码	● 严重	环境变量化
B3-3	弱Secret Key	● 严重	启动时强制拒绝
B3-4	改密绕过	● 高危	✅ W2-1+W2-5已修复
B3-5	CSRF	● 高危	✅ W2-2已修复
B3-6	Logout CSRF	● 中危	GET→POST
B3-7	Cookie属性	● 中危	SameSite=Lax
B3-8	XSS输入	● 低危	白名单校验
B3-9	限流绕过	● 低危	用户名维度
B3-10	版本泄露	● 信息	移除Server头
B3-11	注释残留	● 信息	清除

攻击者用 flask-unsign + 字典爆破即可恢复弱密钥，伪造任意用户 session。

修复：改为 sys.exit(1) 拒绝启动。

[W4-2] Docker -it 模式绕过 isatty 检测 @ app.py:37-38

sys.stdin.isatty() 在 docker run -it 下返回 True，导致 Docker 中仍可开启 DEBUG 模式。

修复：增加 os.path.exists("/.dockerenv") 检测。

[W4-3] 伪造不存在的用户名导致 500 @ app.py + profile.html

get_safe_user("nonexistent") 返回 None，模板 {{ user.username }} 触发 AttributeError。配合 Session 伪造可对指定路由发动 DoS。

修复：在 /profile 和 / 路由中增加 if user is None: session.clear(); return redirect(url_for("login"))。

● 中危 (7 项)

编号	标题	说明	修复
W4-4	多 Worker 限流竞态	内存字典在多进程 Gunicorn 下独立计数	记录为已知限制
W4-5	用户名限流可 DoS 指定用户	user_key 不含 IP 前缀，分布式 IP 可耗尽目标额度	user_key 改为 f"user:{username}:ip:{ip}"
W4-6	日志注入	username 含换行符可伪造日志条目	username.replace('\n','_')
W4-7	CSRF Token 空值绕过	session 无 token 时 compare_digest("", "") 为 True	context_processor 中自动调用 generate_csrf_token()
W4-8	10000条清理误伤	按插入顺序删前一半可能误删活跃用户	记录为已知限制

编号	标题	说明	修复
W4-9	初始密码泄露到 Docker 日志	<code>print()</code> 输出被 <code>docker logs</code> 持久化	已改为 <code>sys.exit(1)</code> 强制设环境变量
W4-10	改密 TOCTOU 竞态	验证原密码与写入之间无锁	记录为已知限制

● 低危 (4 项)

编号	标题	说明
W4-11	角色授权未实现	admin/user 角色已定义但路由无权限区分
W4-12	手机号无长度上限	正则 <code>^\+?\d[\d\-]*\$</code> 接受无限长输入
W4-13	退出链接 405 错误	<code>index.html</code> 中 <code></code> 对 POST-only 端点产生 405
W4-14	改密成功"重新登录"同样 405	<code>change_password.html</code> 同 W4-13

● 信息 (5 项)

编号	标题	说明
W4-15	时序侧信道防御正确	<code>DUMMY_HASH</code> 已实现
W4-16	密码哈希使用 scrypt	Werkzeug 默认选择 (强于 pbkdf2)
W4-17	安全响应头已配置	CSP + XFO + XCTO
W4-18	Session Cookie 属性正确	HttpOnly + SameSite=Lax
W4-19	ProxyFix 已配置	<code>app.wsgi_app = ProxyFix(...)</code> 正确解析反向代理头

黑盒渗透结果

由于 DEBUG=0 且 secret key 不在字典中，黑盒 Agent 未能突破系统。主要发现为信息级别：

- (已确认误报) `/logout` CSRF 保护实际生效 (Agent 只看 302 码未验证副作用)
- Session Cookie Secure 标志建议 (HTTP 测试环境下为 False，属正常)

教训： 黑盒 Agent 需在每次 POST 后验证状态是否真实变更，而非仅依赖 HTTP 状态码。

第四轮总结

本轮自动化架构发现了 3 个真正新增的严重漏洞（#25 弱Key仅警告、#26 Docker绕过isatty、#27 伪造用户500），以及多个对前几轮族系的补充发现和已知限制。关键修复已落地：弱密钥强制 `sys.exit(1)` 拒绝、Docker 模式双重检测、不存在用户空值保护、用户名限流绑定 IP、CSRF Token 空值修复、日志注入过滤。

八、最终代码与全量汇总

修复后项目结构

```
flask-user-mgmt-fixed/
├─ app.py                # 主应用（四轮迭代，566行）
├─ requirements.txt      # Python 依赖
├─ templates/
│   ├─ base.html        # 基础模板（退出POST表单 + CSP头）
│   ├─ login.html       # 登录页（CSRF Token）
│   ├─ index.html       # 首页（时间问候语）
│   ├─ change_password.html # 强制改密/主动改密页
│   └─ profile.html     # 个人信息编辑 + 密码修改页
└─ static/css/
    └─ style.css
```

启动方法

```
cd flask-user-mgmt-fixed
export FLASK_SECRET_KEY="$(python -c 'import os; print(os.urandom(32).hex())')"
export INIT_PWD_ADMIN="YourStrongAdminPwd@2026"
export INIT_PWD_ALICE="YourStrongAlicePwd@2026"
pip install -r requirements.txt
python app.py
```

去重统计

四轮测试共发现 43 个条目，但其中大量是同一根因在不同测试视角下的重复发现。按漏洞族系去重后：

分类	数量	说明
独立漏洞点	12	互不相同的根因
└ 漏洞族系（跨轮继承）	7 族系 / 20 项	同一根因在不同轮次被多次发现
└ 单次独立漏洞	5	仅在一轮中出现，根因独立
已知限制	7	架构取舍或未实现功能，非安全漏洞
信息确认	5	验证防御措施正确，非漏洞
交叉验证	2	黑盒与白盒从不同路径发现同一问题
重复合计	43	

漏洞族系（7 个独立根因，跨 4 轮 20 次发现）

每个族系的核心是一个独立根因。后续轮次发现的"新漏洞"实际上是同一根因在修补后以不同形态残留。

族系 A：Secret Key 管理

轮次	发现	形态	严重程度
R1	#3	代码中硬编码 <code>"dev-key-2025"</code>	严重
R3	#16	部署用弱环境变量 <code>"test-key-2026"</code>	严重
R4	#25	长度校验仅警告不拒绝 → 改为 <code>sys.exit(1)</code>	严重

继承链分析： R1 把硬编码改为 `os.environ.get(..., os.urandom(32).hex())`，修补了"代码写死"问题。但修补漏掉了部署侧——R3 发现运维仍可设置弱值，于是加了长度警告。R4 发现警告不够，运维可能忽略，最终改为拒绝启动。**一家漏洞修了三次才封死。**

族系 B：Debug 模式

轮次	发现	形态	严重程度
R1	#5	代码硬编码 <code>debug=True</code>	严重
R3	#14	部署设 <code>FLASK_DEBUG=1</code> ，Werkzeug 控制台暴露	严重
R4	#26	<code>docker run -it</code> 绕过 <code>isatty()</code> 检测	严重

继承链分析： R1 改为环境变量默认 `False`。R3 发现运维仍可主动设为 `1`，加了交互确认 + 非交互拒绝。R4 发现 Docker `-it` 模式让 `isatty()` 返回 `True`，绕过非交互检测，补了 `/.dockerenv` 检查。

族系 C：密码管理

轮次	发现	形态	严重程度
R1	#4	字典明文存储 + 前端 HTML 展示密码	严重
R3	#15	改哈希后明文仍硬编码在源码 <code>generate_password_hash("Admin@2025#Secure")</code>	严重
R4	#33	随机生成密码经 <code>print()</code> 落入 Docker 日志持久化	中危

继承链分析： R1 加了哈希 + `get_safe_user()` 过滤。但初始密码仍是源码中的明文——R3 发现攻击者读到源码就能用。改为环境变量 `INIT_PWD_*` 后，R4 发现回退到随机生成的密码会 `print()` 到 stdout，被 Docker 日志永久保存。

族系 D：CSRF 防护

轮次	发现	形态	严重程度
R2	#9	所有 POST 表单无 Token	🔴 高危
R3	#19	/logout 为 GET 请求 (Logout CSRF)	🟡 中危
R4	#31	context_processor 注入空串, compare_digest("", "")=True 绕过	🟡 中危

继承链分析： R2 加了 Session-bound CSRF Token。R3 发现 logout 忘了改（仍是 GET `<a>` 链接），改为 POST 表单。R4 发现边缘情况：session 新建时 `csrf_token=""`，表单也是 `""`，`compare_digest` 返回 `True`——空 Token 校验通过。改为 `context_processor` 中主动调用 `generate_csrf_token()`。

族系 E：Session 管理

轮次	发现	形态	严重程度
R2	#8	302 重定向可被拦截，绕过首次改密	🔴 高危
R2	#12	改密后其他设备旧 session 仍有效	🟡 中危
R3	#17	伪造 <code>force_change_password=True</code> 即可跳过原密码验证	🔴 高危（交叉）

三个发现，同一个根因：过度信任客户端 session 中的状态。 R2 #8 修了"其他路由不检查标记"（`before_request` 拦截），R2 #12 修了"密码变更不同步"（`session_version`），然后 R3 黑盒从另一条路径（`secret_key` → 伪造 session → `force_change_password`）证实了同一个信任问题。

族系 F：登录限流

轮次	发现	形态	严重程度
R1	#7	完全无限流	🟡 中危
R2	#10	不同锁类型返回不同消息，泄露阈值	🟡 中危
R3	#22	纯 IP 维度，代理池可绕过	🟢 低危
R4	#29	用户名维度未绑 IP，可被用于 DoS 指定用户	🟡 中危

继承链分析： R1 加了基础 IP 限流。R2 发现消息区分短/长窗口泄露了 3次/60s 和 5次/5min 参数。R3 发现换 IP 就能绕过，加了用户名维度。R4 发现用户名维度不含 IP 前缀，攻击者可故意用分布式 IP 对目标用户名反复失败来锁死该用户。这个族系是典型的"防御-绕过-再防御-再绕过"攻防螺旋。

族系 G：信息泄露

轮次	发现	形态	严重程度
R1	#1	HTML 注释含 <code>admin:admin123</code>	🔴 高危
R3	#23	<code>Server: Werkzeug/3.1.3 Python/3.13.7</code>	🟢 信息
R3	#24	Jinja2 注释含历史凭据记录	🟢 信息
R4	#30	username 含 <code>\n</code> 可伪造日志行	🟡 中危

四个发现共性是"不经意间把不该暴露的信息写出去了"——HTML 输出、HTTP 头、模板文件、日志，四个不同出口。根因相同：没有从攻击者视角审视"输出了什么"。

单次独立漏洞（5 个，仅在一轮中出现）

编号	来源	漏洞	严重程度
#2	R1	弱密码（ <code>admin123/alice2025</code> ）	🔴 高危
#6	R1	绑定 <code>0.0.0.0</code> 过度暴露	🟡 中危
#11	R2	<code>LOGIN_ATTEMPTS</code> 字典条目永不过期 → 内存泄漏	🟡 中危
#13	R2	登录不存在用户跳过哈希 → 时序侧信道	🟢 低危
#20	R3	Session Cookie 缺 <code>SameSite/Secure</code>	🟡 中危
#21	R3	邮箱/手机输入无 XSS 过滤	🟢 低危
#27	R4	伪造不存在用户名 → <code>None.username</code> → 500	🔴 严重

已知限制（7 项，非安全漏洞，属架构取舍或未完成功能）

编号	条目	分类	说明
#28	多 Worker 下内存字典限流失效	架构限制	需 Redis 等共享存储，超出当前实验范围
#32	10000 条清理按插入顺序删前一半	设计取舍	极端场景下的兜底策略，实际很难触发
#34	改密验证与写入间 TOCTOU 窗口	理论风险	单线程 Flask 开发服务器下实际不存在
#35	admin/user 角色未做权限区分	功能未实现	角色字段已定义，路由鉴权未纳入本次需求
#36	手机号正则无长度上限	输入校验完善	可加 <code>maxlength</code> ，优先级低
#37	前端输入校验与后端校验不一致	前后端校验一致	改为后端校验，前端校验仅作提示

37	自页 <code></code> 对 POST- 来自 端点 405	前端 bug 分类	改为 POST 表单后遗留的链接未 漏洞
#38	改密成功页 <code></code> 同 上 405	前端 bug	同 #37

信息确认（5 项，验证防御措施有效）

编号	确认项	结论
#39	时序侧信道 DUMMY_HASH 防御	✅ 实现正确
#40	密码哈希使用 scrypt	✅ 比 pbkdf2 更强
#41	CSP + X-Frame-Options + X-Content-Type-Options	✅ 已配置
#42	Session Cookie HttpOnly + SameSite=Lax	✅ 已配置
#43	ProxyFix 中间件正确解析反向代理头	✅ 已配置

为什么继承漏洞能层层逃脱？

回顾七个族系的演化路径，继承漏洞"逃脱"上一轮修补的模式有三种：

模式一：修了代码，没修部署面。 族系 A（Secret Key）和 B（Debug）最典型。R1 把硬编码改为环境变量，但 R3 发现环境变量的值本身可以是弱/危险的——代码层面的"安全默认值"被部署层面的"不安全覆盖值"击败。**教训：代码安全 ≠ 系统安全，环境变量也是攻击面。**

模式二：修了主路径，漏了边缘路径。 族系 D（CSRF）和 E（Session）属于此类。R2 给全站加了 CSRF Token，但 logout 是 `<a>` 链接不是 `<form>`，被遗漏；session 中 force_change_password 的信任问题从"不检查重定向"蔓延到"伪造 cookie 值"。**教训：安全机制必须覆盖所有入口，一个遗漏就变短板。**

模式三：防御引入新攻击面。 族系 F（限流）最典型。加了用户名维度防御代理池 → 用户名维度本身变成了 DoS 工具。族系 C（密码）同理：改环境变量避免源码泄露 → 随机生成回退的 `print()` 变成 Docker 日志泄露。**教训：每加一层防御，都要审视这层防御本身会不会被滥用。**

第一轮完成于 2026-07-07 | 第二轮（白盒）完成于 2026-07-07

第三轮（黑盒）完成于 2026-07-07 | 第四轮（自动化迭代）完成于 2026-07-07

修复后代码位于 `./flask-user-mgmt-fixed/`